

Práce se soubory: čtení a zápis

Základy programování

Matouš Cejnek
U12110, FS, ČVUT v Praze

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Co si dnes odnesete?

- Bezpečně otevřít a zavřít textový soubor pomocí `with`.
- Zvolit režim čtení, zápisu, přidávání nebo binární režim.
- Zpracovat velký soubor postupně po řádcích nebo blocích.
- Vysvětlit význam UTF-8 a rozdíl mezi textem a bajty.
- Pracovat s cestami pomocí `pathlib` a ošetřit běžné chyby.
- Oddělit načtení, zpracování a uložení výsledků.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

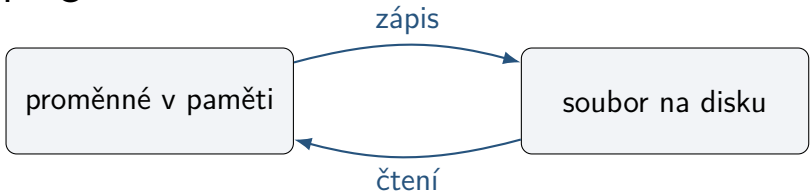
Na co navazujeme?

- Řetězce, seznamy a slovníky uchovávají data **za běhu programu**.
- Funkce oddělují vstup, výpočet a výstup.
- Výjimka umožňuje reagovat na očekávatelný neúspěch.
- Objekt má stav a metody; otevřený soubor je také objekt.

Nová otázka

Jak data zachovat po ukončení programu nebo je předat jinému programu?

Paměť programu není trvalé úložiště



- Proměnné po skončení procesu zmizí.
- Soubor má jméno, umístění a obsah, který může přetrvat.
- Formát souboru je dohoda, jak obsah interpretovat.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Tři různé věci

Cesta určuje, kde soubor hledáme: `data/mereni.txt`.

Soubor je uložená posloupnost bajtů v souborovém systému.

Souborový objekt vznikne voláním `open()` a zprostředkuje práci se souborem.

Pozor na časté nedorozumění

Proměnná `file` neobsahuje celý soubor. Odkazuje na otevřený proud dat.

Životní cyklus práce se souborem



- Otevření může selhat: cesta neexistuje nebo chybí oprávnění.
- Operace musí odpovídat zvolenému režimu.
- Zavření uvolní prostředky a dokončí zápis.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Soubor lze otevřít a zavřít ručně

```
file = open("report.txt", "w", encoding="utf-8")
file.write("Measurement complete.\n")
file.close()
```

- `open()` vrátí objekt, přes který se souborem pracujeme.
- `write()` zapíše data a `close()` práci dokončí.
- Pokud mezi otevřením a zavřením nastane chyba, `close()` se nemusí provést.

Proto obvykle používáme `with`

Zajišťuje zavření automaticky a nemusíme na něj pamatovat v každé větvi programu.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

První bezpečný zápis

```
with open("report.txt", "w", encoding="utf-8") as file:
    file.write("Measurement complete.\n")

print("File is already closed here.")
```

- `open()` vytvoří souborový objekt.
- `as file` uloží odkaz do proměnné.
- Konec bloku `with` soubor automaticky zavře i při chybě.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Režim otevření je součást zadání

Režim	Význam	Když soubor neexistuje
`"r"`	čtení	chyba
`"w"`	zápis od začátku	vytvoří se
`"a"`	přidávání na konec	vytvoří se
`"x"`	vytvoření nového	vytvoří se
`"b"`	přepnutí na bajty	záleží na kombinaci

Režimy lze kombinovat, například `"rb"` nebo `"wb"`.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Nejnebezpečnější písmeno je `w`

Otevření v režimu `"w"` zkrátí existující soubor na nulu

Původní obsah zmizí už při otevření, ještě před prvním voláním `write()`.

- `"w"`: vytváříme nový úplný obsah.
- `"a"`: zachováváme obsah a přidáváme další záznam.
- `"x"`: chceme odmítnout náhodné přepsání existujícího souboru.

Režim vybíráme podle zamýšlené změny, ne podle zvyku.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Trvalý deník používá přidávání

```
from datetime import datetime

message = input("Note: ")
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M")

with open("journal.txt", "a", encoding="utf-8") as file:
    file.write(f"{timestamp} | {message}\n")
```

Každé spuštění přidá jeden samostatný záznam a zachová starší data.

Tři způsoby čtení

```
with open("data.txt", "r", encoding="utf-8") as file:
    all_text = file.read()       # zbytek souboru jako str
    one_line = file.readline()   # jeden dalsi radek
    all_lines = file.readlines() # zbytek jako list[str]
```

- Každá operace pokračuje z **aktuální pozice** v souboru.
- Po `read()` jsme obvykle na konci; další čtení vrátí prázdný výsledek.
- Na velká data je bezpečnější postupné zpracování.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Soubor lze procházet po řádcích

```
with open("measurements.txt", "r", encoding="utf-8") as file:
    for line_number, line in enumerate(file, start=1):
        print(line_number, line.rstrip("\n"))
```

- V paměti je vždy jen malá část souboru.
- Číslo řádku pomůže vysvětlit chybu ve vstupu.
- Stejný princip funguje i pro soubor s miliony řádků.

Konec řádku je také znak

Soubor s obsahem

```
08:00;21.4  \n  09:00;22.1  \n
```

vrací řádky včetně konce řádku.

- `rstrip("\n")` odstraní pouze znak nového řádku.
- `strip()` odstraní i mezery a tabulátory na obou koncích.
- Volba záleží na tom, zda jsou mezery součástí dat.

Prázdný řádek přeskočíme vědomě

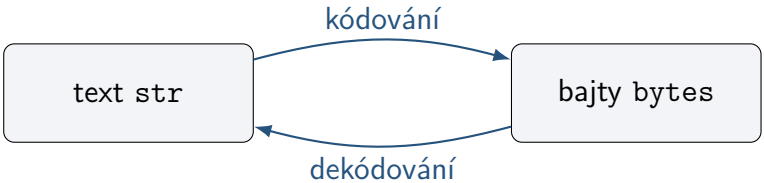
```
with open("measurements.txt", encoding="utf-8") as file:
    for line_number, line in enumerate(file, start=1):
        cleaned = line.strip()
        if not cleaned:
            continue

        print(line_number, cleaned)
```

Důležitý návrh

Rozhodněte, zda je prázdný řádek chyba, oddělovač, nebo položka k přeskočení.

Text a bajty nejsou totéž



- Disk ukládá bajty, Python nám v textovém režimu vrací řetězce.
- `encoding="utf-8"` určuje pravidla převodu.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Stejný znak může zabrat více bajtů

```
text = "caf\u00e9"
data = text.encode("utf-8")

print(len(text))  # 4 znaky
print(len(data))  # 5 bajtu
```

Zápis \u00e9 představuje znak é.

- Délka řetězce není obecně velikost souboru.
- Čtenář i zapisovatel se musí shodnout na kódování.
- UTF-8 je běžná přenositelná volba pro text.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Kódování je součást formátu

- Bez informace o kódování nelze obecné bajty jednoznačně převést na text.
- Chybné kódování může způsobit `UnicodeDecodeError` nebo pokažené znaky.
- `encoding="utf-8"` uvádějte explicitně při čtení i zápisu textu.

Textový soubor

Není „jen text“. Je to posloupnost bajtů interpretovaná podle dohodnutého kódování a formátu.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Práce se souborem může selhat

`FileNotFoundError` vstupní cesta neexistuje.

`PermissionError` proces nemá potřebné oprávnění.

`IsADirectoryError` místo souboru jsme otevřeli složku.

`UnicodeDecodeError` bajty neodpovídají zvolenému kódování.

`ValueError` obsah existuje, ale má neplatný formát pro převod.

Chyba prostředí a chyba obsahu jsou odlišné problémy.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Ošetřujeme očekávatelnou chybu

```
try:
    with open("measurements.txt", encoding="utf-8") as file:
        first_line = file.readline()
except FileNotFoundError:
    print("Input file does not exist.")
else:
    print(first_line.rstrip("\n"))
```

- Blok `try` držte co nejmenší.
- Zachytávejte konkrétní výjimku, kterou umíte smysluplně řešit.
- `except Exception` často skryje chybu v programu.

Chybný řádek nemusí ukončit celý import

```
try:
    temperature = float(value_text)
except ValueError:
    invalid_lines.append(line_number)
    continue

measurements.append((time_text, temperature))
```

- Program zachová platná měření.
- Report zároveň uvede, které řádky přeskočil.
- Tiché ignorování chyb by znehodnotilo výsledek.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Cesty skládáme pomocí pathlib

```
from pathlib import Path

output_dir = Path("outputs")
report_path = output_dir / "report.txt"

output_dir.mkdir(parents=True, exist_ok=True)
report_path.write_text("Done\n", encoding="utf-8")
```

Operátor / skládá části cesty přenositelně a čitelně.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Relativní vůči čemu?

- `Path("data.txt")` je relativní k **aktuální pracovní složce**.
- Pracovní složka nemusí být složka, ve které leží skript.
- `Path.cwd()` ukáže, odkud se relativní cesta vyhodnocuje.
- `path.resolve()` pomůže při diagnostice zobrazit úplnou cestu.

Při chybě „soubor neexistuje“

Nejdříve vypište cestu, kterou program skutečně používá.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Cesta umí popsat soubor

```
from pathlib import Path

path = Path("photos") / "IMG_0042.JPG"

print(path.name)           # IMG_0042.JPG
print(path.stem)           # IMG_0042
print(path.suffix.lower())# .jpg
print(path.exists())
print(path.is_file())
```

Metody pouze zjišťují stav; samy obsah nemění.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

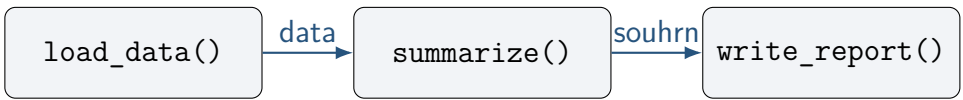
Složku lze procházet bezpečně

```
from pathlib import Path

for path in Path("photos").iterdir():
    if path.is_file() and path.suffix.lower() in {".jpg", ".jpeg"}:
        print(path)
```

- `iterdir()` prochází jednu složku.
- `glob("*.txt")` vybírá podle vzoru.
- `rglob("*")` prochází i podsložky.

Oddělte načtení, zpracování a zápis



- Každá funkce má jednu odpovědnost.
- Výpočet nezávisí na `input()` ani `print()`.
- Stejná data lze zapsat jako textový report nebo SVG.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Řádek nejprve rozložíme, potom převádíme

```
time_text, value_text = cleaned.split(";", maxsplit=1)
temperature = float(value_text)
measurement = (time_text, temperature)
```

Pro vstup `08:00;21.4` vznikne `("08:00", 21.4)`.

- Oddělovač je součást dohody o formátu.
- Počet částí i převod mohou selhat.
- Validaci děláme dříve, než hodnotu použijeme ve výpočtu.

Formát je rozhraní mezi programy

Producent

Zapíše data podle pravidel: oddělovače, pořadí, jednotky, kódování.

Spotřebitel

Čte podle stejných pravidel a odmítá neplatný obsah.

CSV, SVG, HTML i konfigurační soubor jsou příklady takové dohody.

Program může vygenerovat SVG

```
points = "40,140 120,105 200,70 280,90"
svg = f"""<svg xmlns="http://www.w3.org/2000/svg"
                  width="320" height="180">
  <polyline points="{points}" fill="none" stroke="#26537d" />
</svg>
"""

Path("chart.svg").write_text(svg, encoding="utf-8")
```

Prohlížeč přečte text, protože odpovídá formátu SVG.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Binární režim pracuje s bajty

- `"rb"` a `"wb"` neprovádějí převod kódování.
- Čtení vrací `bytes`, zápis očekává `bytes`.
- Obrázky, archivy nebo vlastní číselný formát nejsou obecný text.

Stejné médium, jiná interpretace

Textový i binární soubor jsou na disku bajty. Liší se dohodou, jak je číst.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Číslo lze uložit do pevného počtu bajtů

```
number = 1000
data = number.to_bytes(2, byteorder="little", signed=False)

with open("number.bin", "wb") as file:
    file.write(data)

restored = int.from_bytes(data, byteorder="little", signed=False)
```

Čtenář musí znát počet bajtů, pořadí bajtů a znaménkovost.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Časté chyby

- Otevření existujícího vstupu omylem v režimu `"w"`.
- Chybějící `encoding="utf-8"` u textového souboru.
- Načtení velkého souboru jedním `read()` bez potřeby.
- Záměna relativní cesty za cestu relativní ke skriptu.
- `strip()` odstraní mezery, které mohly být součástí dat.
- Zachycení všech výjimek bez informace o chybném souboru či řádku.
- Smíchání načítání, výpočtu, výpisu a uživatelského vstupu v jedné funkci.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Rychlá kontrola porozumění

1. Kdy zvolíte `"w"` a kdy `"a"`?
2. Proč soubor procházíme po řádcích místo `readlines()`?
3. Co přesně dělá `encoding="utf-8"`?
4. Vůči čemu se vyhodnocuje relativní cesta?
5. Jaký typ vrací čtení v režimu `"rb"`?
6. Kde má být `try`, pokud může být chybné jen jedno číslo na řádku?

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Co využijete na cvičení?

- Čtení po řádcích a statistika textu.
- Přidávání záznamů do trvalého deníku.
- Validace měření a vytvoření reportu.
- Bezpečné cesty a dávkové přejmenování fotografií.
- Uložení čísel v textové a binární podobě.
- Sloučení a seřazení měření z více vstupních souborů.
- Generování SVG grafu jako rozšiřující výzva.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Shrnutí

- `with` bezpečně spravuje životní cyklus otevřeného souboru.
- Režim určuje povolené operace i riziko přepsání dat.
- Text převádíme mezi `str` a bajty pomocí kódování.
- Velká data zpracováváme postupně po řádcích nebo blocích.
- `pathlib` skládá cesty a zpřístupňuje informace o souborech.
- Robustní program oddělí načtení, zpracování a zápis a hlásí chyby s kontextem.